

```

Customer::Customer(const Customer& rhs)
    : name(rhs.name)                //复制 rhs 的数据
{
    logCall("Customer copy constructor");
}

Customer& Customer::operator=(const Customer& rhs)
{
    logCall("Customer copy assignment operator");
    name = rhs.name;                //复制 rhs 的数据
    return *this;                   //见条款 10
}

```

这里的每一件事情看起来都很好，而实际上每件事情也的确都好，直到另一个成员变量加入战局：

```

class Date { ... };                //日期
class Customer {
public:
    ...                            //同前
private:
    std::string name;
    Date lastTransaction;
};

```

这时候既有的 *copying* 函数执行的是局部拷贝（*partial copy*）：它们的确复制了顾客的 *name*，但没有复制新添加的 *lastTransaction*。大多数编译器对此不出任何怨言——即使在最高警告级别中（见条款 53）。这是编译器对“你自己写出 *copying* 函数”的复仇行为：既然你拒绝它们为你写出 *copying* 函数，如果你的代码不完全，它们也不告诉你。结论很明显：如果你为 *class* 添加一个成员变量，你必须同时修改 *copying* 函数。（你也需要修改 *class* 的所有构造函数（见条款 4 和条款 45）以及任何非标准形式的 *operator=*（条款 10 有个例子）。如果你忘记，编译器不太可能提醒你。）

一旦发生继承，可能会造成此一主题最暗中肆虐的一个潜藏危机。试考虑：

```

class PriorityCustomer: public Customer {    //一个 derived class
public:
    ...
    PriorityCustomer(const PriorityCustomer& rhs);
    PriorityCustomer& operator=(const PriorityCustomer& rhs);
    ...
private:
    int priority;
};

```

```

PriorityCustomer::PriorityCustomer(const PriorityCustomer& rhs)
    : priority(rhs.priority)
{
    logCall("PriorityCustomer copy constructor");
}

PriorityCustomer&
PriorityCustomer::operator=(const PriorityCustomer& rhs)
{
    logCall("PriorityCustomer copy assignment operator");
    priority = rhs.priority;
    return *this;
}

```

PriorityCustomer 的 *copying* 函数看起来好像复制了 PriorityCustomer 内的每一样东西, 但是请再看一眼。是的, 它们复制了 PriorityCustomer 声明的成员变量, 但每个 PriorityCustomer 还内含它所继承的 Customer 成员变量复件 (副本), 而那些成员变量却未被复制。PriorityCustomer 的 *copy* 构造函数并没有指定实参传给其 base class 构造函数 (也就是说它在它的成员初值列 (member initialization list) 中没有提到 Customer), 因此 PriorityCustomer 对象的 Customer 成分会被不带实参之 Customer 构造函数 (即 *default* 构造函数——必定有一个否则无法通过编译) 初始化。*default* 构造函数将针对 name 和 lastTransaction 执行缺省的初始化动作。

以上事态在 PriorityCustomer 的 *copy assignment* 操作符身上只有轻微不同。它不曾企图修改其 base class 的成员变量, 所以那些成员变量保持不变。

任何时候只要你承担起“为 derived class 撰写 *copying* 函数”的重责大任, 必须很小心地也复制其 base class 成分。那些成分往往是 *private* (见条款 22), 所以你不能直接访问它们, 你应该让 derived class 的 *copying* 函数调用相应的 base class 函数:

```

PriorityCustomer::PriorityCustomer(const PriorityCustomer& rhs)
    : Customer(rhs), //调用base class 的 copy 构造函数
      priority(rhs.priority)
{
    logCall("PriorityCustomer copy constructor");
}

PriorityCustomer&
PriorityCustomer::operator=(const PriorityCustomer& rhs)
{
    logCall("PriorityCustomer copy assignment operator");
    Customer::operator=(rhs); //对base class 成分进行赋值动作
    priority = rhs.priority;
    return *this;
}

```

本条款题目所说的“复制每一个成分”现在应该很清楚了。当你编写一个 *copying* 函数，请确保 (1) 复制所有 local 成员变量，(2) 调用所有 base classes 内的适当的 *copying* 函数。

这两个 *copying* 函数往往有近似相同的实现本体，这可能会诱使你让某个函数调用另一个函数以避免代码重复。这样精益求精的态度值得赞赏，但是令某个 *copying* 函数调用另一个 *copying* 函数却无法让你达到你想要的目标。

令 *copy assignment* 操作符调用 *copy* 构造函数是不合理的，因为这就像试图构造一个已经存在的对象。这件事如此荒谬，乃至根本没有相关语法。是有一些看似如你所愿的语法，但其实不是；也的确有些语法背后真正做了它，但它们在某些情况下会造成你的对象败坏，所以我不打算将那些语法呈现给你看。单纯地接受这个叙述吧：你不该令 *copy assignment* 操作符调用 *copy* 构造函数。

反方向——令 *copy* 构造函数调用 *copy assignment* 操作符——同样无意义。构造函数用来初始化新对象，而 *assignment* 操作符只施行于已初始化对象身上。对一个尚未构造好的对象赋值，就像在一个尚未初始化的对象身上做“只对已初始化对象才有意义”的事一样。无聊嘛！别尝试。

如果你发现你的 *copy* 构造函数和 *copy assignment* 操作符有相近的代码，消除重复代码的做法是，建立一个新的成员函数给两者调用。这样的函数往往是 *private* 而且常被命名为 *init*。这个策略可以安全消除 *copy* 构造函数和 *copy assignment* 操作符之间的代码重复。

请记住

- *Copying* 函数应该确保复制“对象内的所有成员变量”及“所有 base class 成分”。
- 不要尝试以某个 *copying* 函数实现另一个 *copying* 函数。应该将共同机能放进第三个函数中，并由两个 *copying* 函数共同调用。

3

资源管理

Resource Management

所谓资源就是，一旦用了它，将来必须还给系统。如果不这样，糟糕的事情就会发生。C++ 程序中最常使用的资源就是动态分配内存（如果你分配内存却从来不曾归还它，会导致内存泄漏），但内存只是你必须管理的众多资源之一。其他常见的资源还包括文件描述器（file descriptors）、互斥锁（mutex locks）、图形界面中的字型 and 笔刷、数据库连接、以及网络 sockets。不论哪一种资源，重要的是，当你不再使用它时，必须将它还给系统。

尝试在任何运用情况下都确保以上所言，是件困难的事，但当你考虑到异常、函数内多重回传路径、程序维护员改动软件却没能充分理解随之而来的冲击，态势就很明显了：资源管理的特殊手段还不很充分够用。

本章一开始是一个直接而易懂且基于对象（object-based）的资源管理办法，建立在 C++ 对构造函数、析构函数、*copying* 函数的基础上。经验显示，经过训练后严守这些做法，可以几乎消除资源管理问题。然后本章的某些条款将专门用来对付内存管理。这些排列在后的专属条款弥补了先前一般化条款的不足，因为管理内存的那个对象必须知道如何适当而正确地工作。

条款 13: 以对象管理资源

Use objects to manage resources.

假设我们使用一个用来塑模投资行为（例如股票、债券等等）的程序库，其中各式各样的投资类型继承自一个 root class `Investment::`

```
class Investment { ... };           // “投资类型” 继承体系中的 root class
```

进一步假设，这个程序库系通过一个工厂函数（factory function，见条款 7）供应我们某特定的 Investment 对象：

```
Investment* createInvestment(); //返回指针，指向 Investment 继承体系内
                                //的动态分配对象。调用者有责任删除它。
                                //这里为了简化，刻意不写参数。
```

一如以上注释所言，createInvestment 的调用端使用了函数返回的对象后，有责任删除之。现在考虑有个 f 函数履行了这个责任：

```
void f()
{
    Investment* pInv = createInvestment(); //调用 factory 函数
    ...
    delete pInv;                          //释放 pInv 所指对象
}
```

这看起来妥当，但若干情况下 f 可能无法删除它得自 createInvestment 的投资对象——或许因为 “...” 区域内的一个过早的 return 语句。如果这样一个 return 被执行起来，控制流就绝不会触及 delete 语句。类似情况发生在对 createInvestment 的使用及 delete 动作位于某循环内，而该循环由于某个 continue 或 goto 语句过早退出。最后一种可能是 “...” 区域内的语句抛出异常，果真如此控制流将再次不会幸临 delete。无论 delete 如何被略过去，我们泄漏的不只是内含投资对象的那块内存，还包括那些投资对象所保存的任何资源。

当然啦，谨慎地编写程序可以防止这一类错误，但你必须想想，代码可能会在时间渐渐过去后被修改。一旦软件开始接受维护，可能会有某些人添加 return 语句或 continue 语句而未能全然领悟它对函数的资源管理策略造成的后果。更糟的是 f 的 “...” 区域有可能调用一个“过去从未抛出异常，却在被‘改善’之后开始那么做”的函数。因此单纯倚赖“f 总是会执行其 delete 语句”是行不通的。

为确保 createInvestment 返回的资源总是被释放，我们需要将资源放进对象内，当控制流离开 f，该对象的析构函数会自动释放那些资源。实际上这正是隐身于本条款背后的半边想法：把资源放进对象内，我们便可倚赖 C++ 的“析构函数自动调用机制”确保资源被释放。（稍后讨论另半边想法。）